

HARBORFLOW | COMPLETE PROJECT MAP

[DIT014 Assignment 2](#) | [Team discussion guide](#) | [Read top to bottom](#)

1. SETUP & TEAM AGREEMENT

Inspect starter package and DATA-DICTIONARY.md. Keep source data unchanged.
Agree on function inputs / returns, feature owners and reviewers.



2. main.py | TASK 0: MENU & USER INPUT

Validate menu choice 1-10. Services 1-9 return to menu after completion or recoverable errors. Repeat only required prompts. Option 10 sets running = False and prints closing message. Use a loop-state variable; do not stop the menu with exit() or break().



3. data_access.py | SHARED LOADING & VALIDATION

Read JSON / CSV using relative paths. Validate fields, dates and numeric values where required. Report expected problems; skip bad records where the service requires it. Catch specific exceptions. Use Python standard library only. No hard-coded sample records or blanket except clauses.



4. DATA RELATIONSHIPS

```
vessels/*.json --vessel_id --> port_calls.csv --call_id --> manifests/*.json
manifests/*.json --shipment_ids --> shipments/*.json
port_calls.csv --call_id --> incidents/*.json
port_calls.csv --port_code + arrival_date --> weather.csv
Read IDs inside records: filenames are not guaranteed to equal business IDs.
```



5. SERVICES | Operational logic + text and algorithms

Proposed modules: operations.py for Tasks 1-5, 8; text_tools.py for Tasks 6, 7, 9. main.py handles interaction.

TASK 1 | LIST VESSELS

- Require vessel_id, name, imo, capacity_teu.
- Report and skip invalid files.
- Sort names case-insensitively.
- Print vessels and skipped-file count.

TASK 2 | INSPECT MANIFEST

Vessel + reference date. Earliest qualifying call; tie: smallest call_id. Load manifest. Resolve shipments in manifest order. Print cargo, counts, weight and missing IDs.

TASK 3 | PRIORITY CARGO

Call ID. FIRST matching rule:
Hazardous > temperature controlled >
EXPRESS or weight >= 5000 > standard.
Print category, reason and ordered totals.

TASK 4 | CUSTOMER PROFILE

Match customer across valid shipments.
Count, weight, express count, destinations.
Append new row or replace existing row.
Preserve others; exact header, 2 decimals.

TASK 5 | CALLS BY MONTH

Validate integer month 1-12.
Skip invalid dates / unknown vessels.
Sort by arrival date, then vessel name.
Print matching calls or no-results message.

TASK 6 | SANITIZE REPORT

- Sort titles and validate numbered selection.
- re.sub: container IDs, seals, emails.
- Preserve other text and line breaks.
- Save only when a substitution occurs.

TASK 7 | STABLE SEQUENCE

Manifest event_codes; sliding window.
Track start and last-seen positions.
Longest contiguous sequence: no repeats.
Equal maximum lengths keep the first.

TASK 8 | WEATHER RISK

ALL upcoming calls; sort date then call_id.
Match forecast by port + date. Apply ALL
true recommendations in required order.
Missing forecast: report and continue.

TASK 9 | INCIDENT SEARCH

Normalize words; index + title lookup.
Rebuild missing / bad / outdated cache.
Score DISTINCT matching query words.
Sort score descending, then incident ID.



6. RESULTS & PERSISTENT FILES

Console: exact prompts, punctuation, capitalization and ordering. Return to menu after service.
 Task 4: exports/customer-profiles.csv | append / update without duplicate customer rows.
 Task 6: exports/sanitized/<incident_id>-sanitized.txt | create only after a substitution.
 Task 9: dataset/incident-index.json | valid JSON, sorted keys / ID lists, reusable cache.



7. INTEGRATION & ACCEPTANCE TESTING

Owner implements; second member reviews; integrate continuously and run through actual menu.
Test normal, boundary, missing-data and malformed-data cases. Record evidence in test-ledger.csv.
Check: rule precedence, exact thresholds, regex no-match, sequence repeats / ties, repeated query words.
Check: first export, second customer, existing-row update, reload CSV / JSON, corrupt-cache rebuild.



8. FINAL HANDOVER

Run from clean folder with fresh supplied dataset and relative paths. Demonstrate all Tasks 0-9. Verify exact outputs; remove debug prints and temporary test data.

readme.txt: members, concrete contributions, architecture, run instructions and limitations.

Submit main.py, all imported Python modules, readme.txt and test-ledger.csv to correct CodeGrade group.

Every member explains one end-to-end flow. All tasks compulsory; assessment is G / U.

Project overview and shared foundations

Purpose. One terminal-based Python application connects fragmented port data and gives users auditable operational answers. All Tasks 0-9 are compulsory. The group must contain 3-4 students; assessment is G/U. This guide explains the complete map on page 1. It supports the official assignment rather than replacing it.

Start with the supplied data

Extract the starter package, preserve its directory structure, and read DATA-DICTIONARY.md before implementation. Inspect actual records, including intentionally malformed and missing records. Do not repair or delete them. The supplied PDF does not contain the complete data dictionary or a submission deadline; check the starter package and Canvas.

Agree on responsibilities and interfaces

Recommended organization: `main.py` for menu and interaction; `data_access.py` for paths, JSON/CSV loading, validation and file writing; `operations.py` for operational calculations; `text_tools.py` for sanitization, sequence logic and search. These boundaries and function names are proposals, not mandatory APIs. A service may use several modules.

For each shared function, agree on its name, inputs, return format and error behavior. A useful collection loader can return valid records plus rejected-record information. Avoid duplicate loaders and duplicate error messages. Keep classification and other calculations independent of input prompts and file access where practical.

Understand the relationships

A vessel is a ship; a port call is one scheduled visit by that ship. `vessel_id` connects vessels to calls. `call_id` connects calls to manifests and incidents. `shipment_ids` in a manifest identify cargo records. Weather requires both `port_code` and `arrival_date` to match. Read and compare identifiers inside files; filenames cannot be assumed to equal business IDs.

Task 0: the shell and recovery

Display the exact required menu. Reject non-integer and out-of-range choices with **Error - Select a service from 1 to 10**. Use a loop-state variable. Do not use `exit()` or `break` to stop the menu. Services return to the menu after completion or recoverable errors. Option 10 ends the loop and prints **Console closed. HarborFlow operational data remains safe**.

Catch `FileNotFoundError`, `json.JSONDecodeError`, `csv.Error` and `ValueError` only around operations where a useful response is possible. Do not use a blanket except that hides programming defects. Use standard-library modules such as `json`, `csv`, `re`, `pathlib` and `datetime`; third-party packages such as `pandas` are prohibited.

Operational services: Tasks 1-5

Task 1: list registered vessels

Read every .json file in dataset/vessels. Require vessel_id, name, imo and capacity_teu. Report and skip malformed or incomplete records while allowing valid vessels to appear. Sort valid vessels by name case-insensitively. Print the required vessel lines and skipped-file count; preserve the specified formatting, including capacity separators.

Task 2: inspect a vessel manifest

Display vessels, request a vessel name and reference date, trim surrounding name spaces and match case-insensitively. Choose the earliest call on or after the reference date; if dates tie, choose the alphabetically smallest call_id. Find the matching manifest through call_id and resolve shipment IDs in manifest order.

Show missing shipment records as **MISSING RECORD**. Declared shipments is the number of entries in the manifest; resolved weight sums only available valid shipment weights. Count missing records separately. Missing hazardous or temperature_controlled flags count as false; missing cargo_type displays **NOT PROVIDED**. Format dates using datetime as day month year.

An invalid date prints **Error - Date must use YYYY-MM-DD.** and repeats only the date prompt. Unknown vessel: **Vessel not found.** No qualifying call: **No upcoming port calls found.** No readable valid manifest: **Manifest unavailable.** These three outcomes return to the menu.

Task 3: identify priority cargo

Request a call ID, trim spaces and match case-insensitively. Unknown call prints **Port call not found.**; unavailable manifest prints **Manifest unavailable.** Classify each resolved shipment using the FIRST matching rule: hazardous -> CONTROLLED; temperature_controlled -> COLD CHAIN; EXPRESS priority or weight_kg >= 5000 -> EXPEDITE; otherwise STANDARD.

A hazardous express shipment is CONTROLLED because rule order is policy. Print category and reason in manifest order. Print totals in this order: CONTROLLED, COLD CHAIN, EXPEDITE, STANDARD. A pure classification function should return both category and reason. Check official examples for exact required wording; do not invent undocumented behavior for ambiguous cases.

Task 4: export a customer profile

Match a trimmed customer name case-insensitively across all valid shipments. Calculate shipment_count, total_weight_kg, express_count and unique destinations. Sort destination codes and join with |. Use exactly two decimals for total weight. The CSV header is exactly **customer,shipment_count,total_weight_kg,express_count,destinations.**

Write exports/customer-profiles.csv. Add a new customer or replace an existing row, preserving other customers and avoiding duplicates. Write the header once. Read rows into memory before updating and rewriting. If no shipments match, print **Customer not found.** and do not create or modify the export. Success prints **Customer profile updated: exports/customer-profiles.csv.**

Task 5: find calls by month

Request an integer month 1-12. Invalid input prints **Error - Enter a month from 1 to 12.** and repeats only the month prompt. Resolve vessel names, report and skip invalid dates and unknown vessel links. Select calls in the requested month and sort by arrival_date then vessel name. No valid matches prints **No port calls found.**

Text, sequences and weather: Tasks 6-8

Task 6: sanitize an incident report

Sort incident titles case-insensitively and number them from 1. Invalid selection prints **Error - Select a listed incident.** and repeats only the selection prompt. Use `re.sub` for three transformations: four letters followed by seven digits -> [CONTAINER]; SEAL- followed by six digits -> [SEAL]; assignment-defined email addresses -> [EMAIL]. Container letters and the SEAL prefix match case-insensitively.

For email matching, the local part permits letters, digits and `._%+-`; the domain permits letters, digits, dots and hyphens, with a final dot followed by at least two letters. Preserve other text, punctuation and line breaks. Save `exports/sanitized/<incident_id>-sanitized.txt` only if a substitution occurs; create the directory when necessary. Otherwise print **Incident contains no share-sensitive identifiers.**

Specification question: the PDF example appears to omit the ordinary word "Container" from its sanitized result, while the explicit requirement says to preserve all other text. Follow the preservation requirement and confirm this example inconsistency with the teacher.

Task 7: longest stable event sequence

For a selected call, use the manifest `event_codes` list in chronological order. Find the longest contiguous sequence without repeated event codes. Contiguous means adjacent original elements; do not remove intermediate events to assemble a result. Implement the sliding window yourself.

Track the window start and each event's last position. A repeat inside the active window moves the start just beyond that previous position. A previous occurrence outside the active window does not move the start. Update the best result only when the current window is strictly longer, preserving the first maximum when lengths tie.

Example: GATE_IN, SCAN, WEIGH, SCAN, LOAD, SEAL, DEPART produces length 5 and WEIGH -> SCAN -> LOAD -> SEAL -> DEPART. Test all-unique events, immediate repeats, repeats outside the window and ties. The PDF is less explicit about Task 7 prompts and failure messages; check starter guidance before choosing extra interface text.

Task 8: assess weather risk

Request a trimmed, case-insensitive vessel name and valid YYYY-MM-DD reference date. Unknown vessel returns to the menu; invalid date repeats only the date prompt. Report and skip invalid call dates. Unlike Task 2, select ALL calls on or after the date. Sort by date then `call_id` and match weather by port code and date.

Apply ALL true recommendations in table order: `wind_kmh >= 45` -> Suspend crane operations.; `wave_height_m >= 2.5` -> Arrange tug assistance.; `precipitation_mm >= 8` and `wind_kmh < 45` -> Prepare covered handling.; all three below thresholds -> Normal operating plan.

A missing forecast prints **Forecast unavailable** and does not stop later calls. Convert numeric CSV fields in a focused try block and report malformed forecast rows. No valid upcoming calls prints **No upcoming port calls found.** Test exact thresholds, combined recommendations and missing forecasts.

Search, integration and final delivery

Task 9: build and reuse the search index

Normalize incident report text into lowercase words using regex to remove punctuation. Build an inverted index mapping words to incident-ID lists and a title lookup mapping incident IDs to titles. A repeated word in one report must not duplicate its incident ID in that word's list. Save dataset/incident-index.json with top-level keys **index** and **titles**; sort word keys, title keys and every incident-ID list.

Rebuild a missing, malformed or outdated cache. It is outdated when it is older than any source incident file. Otherwise load and reuse it. Normalize queries using exactly the same rules. Score by distinct query words matching each incident, retain only positive scores, and sort by descending score then incident ID. Display cached titles. Searching must use the loaded index and title lookup, rather than rereading source report text.

Continuous integration and meaningful evidence

Give each feature an owner and a second-member reviewer. Review normal behavior and failure paths, integrate continuously into main.py, and test through the real menu. Record inputs, expected outcomes and actual outcomes in test-ledger.csv. Use any ledger format provided by the starter package; otherwise agree a shared format.

Minimum evidence includes missing files, malformed JSON, invalid CSV numbers, mixed valid/invalid records, known and unknown IDs, missing linked shipments, first exports, second customers, existing-row updates and reloading outputs. Also cover each sensitive identifier, multiple identifiers and no-match reports; sequence repeats and ties; weather thresholds and combined rules; single-word searches, repeated query words, no matches and corrupt-cache rebuilds.

Definition of done and CodeGrade submission

Run from a clean folder using a fresh unchanged supplied dataset. Check every service, exact prompts, punctuation, capitalization, ordering and output filenames. Remove debug prints, temporary test data and machine-specific absolute paths. Generated CSV/JSON must remain valid after repeated runs. Do not submit altered dataset copies unless CodeGrade explicitly requests them.

Submit main.py and every imported Python module, readme.txt and test-ledger.csv. README content: member names, concrete contributions, architecture, run instructions and known limitations. Use the correct CodeGrade group containing exactly 3-4 students. Every member must understand the integrated product and explain an end-to-end flow. Review feedback and resubmit before the deadline when allowed.

Recommended first milestone

Agree on loader return formats and error ownership, then build a working menu and vessel listing that handles invalid files. This gives the team an end-to-end foundation: input -> loading -> validation -> processing -> output -> menu. Balance work by complexity rather than task count; loaders and search indexing may require additional support.

Source and status: Based on the supplied DIT014 HT26 Assignment 2 - HarborFlow Port Intelligence Console PDF, version 19 September 2026. This document is a team explanation, not an official solution, implementation or test report. No tests are claimed to have been run.

One continuous meeting explanation

Hi everyone. I will use this diagram to explain the whole project, from setup to submission. We are building one Python terminal application that connects vessel records, schedules, manifests, shipments, incidents and weather forecasts.

We start at the top by inspecting the starter package and reading the data dictionary. Before working separately, we agree on shared function names, inputs, return values and error behavior. We also assign feature owners and reviewers.

The application starts in `main.py`. The menu validates a selection from one to ten. Services one to nine return to the menu when they finish, or after a recoverable error. Option ten changes the loop-state variable and closes the console. We do not use `exit` or `break` to stop the menu.

The next layer is shared data access. Our loaders read JSON and CSV, validate required fields and check dates and numeric values where needed. Expected data problems produce useful messages. A bad record should not prevent valid records from being processed when the service allows us to continue.

In the data section, `vessel_id` connects a ship to its port calls. A port call is one scheduled visit, so the same ship can have several calls. `call_id` connects a visit to its manifest and incidents. The manifest contains shipment IDs. Weather is matched using both the port code and arrival date. We use identifiers inside the records rather than assume filenames match them.

Under operational logic, task one lists valid vessels in name order and reports skipped files. Task two selects the earliest call on or after a reference date. It resolves shipments in manifest order, lists missing records separately and sums only resolved weights. If the earliest dates tie, the smallest call ID is selected.

Task three applies the first matching cargo rule: hazardous, then temperature controlled, then express or heavy, then standard. Each shipment receives one category and a reason. This is different from weather assessment, where several recommendations can apply together.

Task four summarizes a customer across valid shipments. It creates or updates one CSV row without duplicating the customer or losing other rows. Destinations are unique and sorted, and weight has two decimals. Task five lists valid calls for a month, sorted by date and vessel name.

Task eight processes all upcoming calls for a vessel, not just the earliest one. It matches forecasts by port and date. Wind of at least forty-five means suspending crane operations; waves of at least two point five mean arranging tug assistance; rain of at least eight with wind below forty-five means covered handling. Applicable recommendations are combined in the required order. Missing forecasts do not stop later calls.

Under text and algorithms, task six replaces container IDs, seals and emails using regular expressions. Other text and line breaks remain unchanged. It saves a file only if a replacement occurs. Task seven finds the longest contiguous event sequence without repetition using a sliding window and last-seen positions. When maximum lengths tie, it keeps the first sequence.

Task nine builds an inverted index that maps words to incident IDs, plus a lookup for titles. We reuse a valid current cache and rebuild a missing, malformed or outdated one. Search counts distinct matching query words, so repeated query words do not increase the score. Results are sorted by descending score and then incident ID.

At the output layer, we have console results, customer CSV profiles, sanitized text reports and the JSON search index. Saved files must remain valid on later runs. Exact messages, punctuation, ordering and filenames are part of the requirements.

Alongside implementation, a second member reviews each feature and we integrate continuously. We record normal, boundary, missing-data and malformed-data tests in the ledger. We also check repeated runs, customer updates, cache rebuilding, exact thresholds and sequence ties.

Finally, we run everything from a clean folder with the supplied dataset, verify all tasks and finish the README. We submit every required Python module, the README and the test ledger to the correct CodeGrade group. Every member should explain one complete data flow. My suggestion is to agree on interfaces first and use the menu and vessel listing as our first integrated milestone.